

1. (0,5) Baseando-se na DDL, crie um banco de dados (*database*) com as seguintes especificações:

- a. Nome: TRABALHO_AVALIATIVO;
- b. Codificação: WIN1252 e ou LANTIN1
- c. Modelo (template): template0
- d. Número de conexão: ilimitado

```
CREATE DATABASE "TRABALHO_AVALIATIVO"
TEMPLATE = template0
ENCODING 'WIN1252'
CONNECTION LIMIT -1;
```

2. (1,0) Tendo como base o esquema de Banco de Dados relacional abaixo representado pela Figura 1, e fazendo uso da DDL, crie de maneira e ordem correta TODAS as tabelas.

OBSERVAÇÃO: Para as restrições de *chaves-primária* e *chaves-estrangeira*, utilize *identificadores* explicitamente.

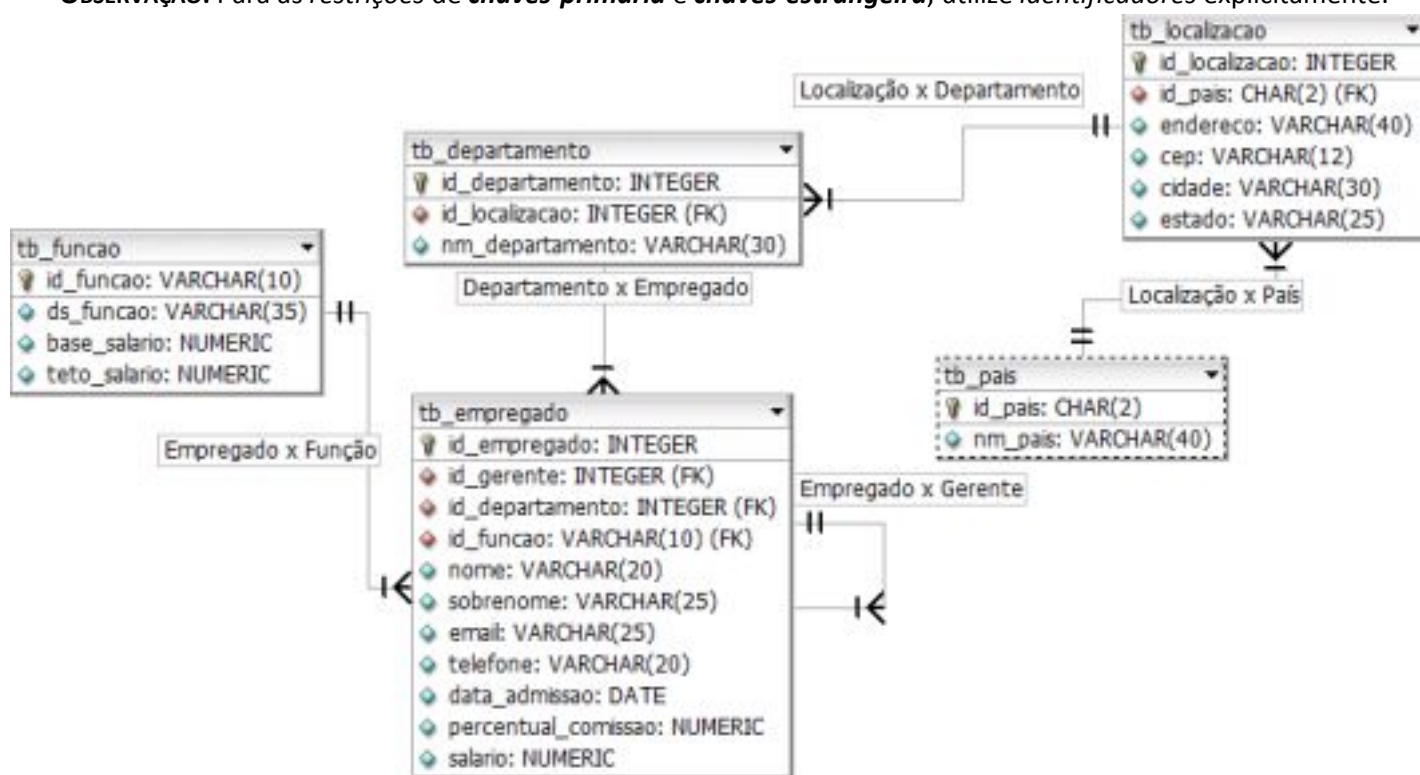


Figura 1 – Esquema de Banco de Dados Relacional

```
CREATE TABLE tb_pais(
id_pais CHAR(2) CONSTRAINT nn_id_pais NOT NULL,
nm_pais VARCHAR2(40),
CONSTRAINT pk_id_pais PRIMARY KEY (id_pais));
```

```
CREATE TABLE tb_localizacao(
id_localizacao NUMBER(4),
```

```
id_pais CHAR(2),
endereco VARCHAR2(40),
cep VARCHAR2(12),
cidade VARCHAR2(30) CONSTRAINT nn_loc_cidade NOT NULL,
estado VARCHAR2(25));
CREATE UNIQUE INDEX pk_id_localizacao
ON tb_localizacao (id_localizacao);
ADD (CONSTRAINT pk_id_loc
PRIMARY KEY (id_localizacao),
CONSTRAINT fk_id_pais
FOREIGN KEY (id_pais)
REFERENCES tb_pais(id_pais));
```

```
CREATE TABLE tb_departamento(
id_departamento NUMBER(4),
nm_departamento VARCHAR2(30) CONSTRAINT nn_nm_depto NOT NULL,
id_localizacao NUMBER(4));
CREATE UNIQUE INDEX pk_id_departamento
ON tb_departamento (id_departamento);
ADD (CONSTRAINT pk_id_departamento
PRIMARY KEY (id_departamento),
CONSTRAINT fk_loc_departamento
FOREIGN KEY (id_localizacao) REFERENCES tb_localizacao (id_localizacao));
```

```
CREATE TABLE tb_funcao(
id_funcao VARCHAR2(10),
ds_funcao VARCHAR2(35) CONSTRAINT nn_ds_funcao NOT NULL,
base_salario NUMBER(8,2),
teto_salario NUMBER(8,2));
CREATE UNIQUE INDEX pk_id_funcao
ON tb_funcao (id_funcao);
ADD (CONSTRAINT pk_id_funcao
PRIMARY KEY(id_funcao));
```

```
CREATE TABLE tb_employado(
id_employado NUMBER(6),
nome VARCHAR2(20),
sobrenome VARCHAR2(25) CONSTRAINT nn_emp_sobrenome NOT NULL,
email VARCHAR2(25) CONSTRAINT nn_emp_email NOT NULL,
telefone VARCHAR2(20),
data_admissao DATE CONSTRAINT nn_emp_dt_adm NOT NULL,
salario NUMBER(8,2),
percentual_comissao NUMBER(2,2),
id_gerente NUMBER(6),
id_departamento NUMBER(4),
id_funcao VARCHAR2(10) CONSTRAINT nn_emp_funcao NOT NULL,
CREATE UNIQUE INDEX pk_id_emp
ON tb_employado (id_employado);
ADD (CONSTRAINT pk_id_emp
PRIMARY KEY (id_employado),
CONSTRAINT fk_emp_depto
FOREIGN KEY (id_departamento) REFERENCES tb_departamento,
CONSTRAINT fk_emp_funcao
```

```

FOREIGN KEY (id_funcao) REFERENCES tb_funcao (id_funcao),
CONSTRAINT fk_emp_gerente
FOREIGN KEY (id_gerente) REFERENCES tb_empregado);
ALTER TABLE tb_departamento
ADD (CONSTRAINT fk_gerente_depto
FOREIGN KEY (id_gerente) REFERENCES tb_empregado (id_empregado));

```

3. (0,5) Elabore uma consulta para exibir o nome completo do empregado, sua respectiva descrição da função e a data de admissão dos empregados admitidos entre o período de 20 de fevereiro de 1987 e 1 de maio de 1989. Ordene a consulta resultante de modo ascendente de maneira posicional pela data de admissão.

```

SELECT e.nome || ' ' || e.sobrenome AS "Nome Completo", f.ds_funcao, e.data_emissao
FROM tb_empregado e
INNER JOIN tb_funcao f ON(e.id_funcao = f.id_funcao)
WHERE e.data_admissao BETWEEN '20/02/1987' AND '01/05/1989'
ORDER BY 3 asc;

```

4. (1,0) Elabore uma consulta para exibir o nome completo do empregado, nome do departamento e nome do país, para todos os empregados cujo nome inicia-se pelos caracteres B, L ou A. Forneça um *label* apropriado para cada coluna.

```

SELECT e.nome || ' ' || e.sobrenome AS "Nome Completo",
       d.nome_departamento AS "Departamento Funcional",
       p.nm_pais AS "Nome país"
FROM tb_empregado e
INNER JOIN tb_departamento d ON(e.id_departamento = d.id_departamento)
INNER JOIN tb_localizacao l ON(d.id_localizacao = l.id_localizacao)
INNER JOIN tb_pais p ON(l.id_pais = p.id_pais)
WHERE e.nome LIKE 'B%'
OR e.nome LIKE 'L%'
OR e.nome LIKE 'A%';

```

5. (1,0) Elabore uma consulta para exibir o nome do empregado, o nome do departamento e sua respectiva localização (cidade e estado) de todos os empregados que recebem comissão.

```

SELECT e.nome, d.nm_departamento, l.cidade, l.estado
FROM tb_empregado e
INNER JOIN tb_departamento d ON (e.id_departamento = d.id_departamento)
INNER JOIN tb_localizacao l ON (d.id_localizacao = l.id_localizacao)
WHERE e.percentual_comissao IS NOT NULL;

```

6. (1,0) Realize uma *Auto Junção* para recuperar o *nome* de cada empregado juntamente com o *nome* de seu respectivo gerente. **Exemplo:** João *trabalha para o* Tiago.

Para o empregado que NÃO possuir gerente vinculado, utilize a função apropriada do *PostgreSQL* substituindo o valor nulo (NULL) para o STRING “*os acionistas*”. Ordene de maneira descendente à relação resultante pelo NOME do gerente.

```

SELECT e.nome || ' trabalha para ' || COALESCE(g.nome, 'Os Acionistas')
FROM tb_empregado e
LEFT OUTER JOIN tb_empregado g ON(e.id_gerente = g.id_empregado)
ORDER BY g.nome DESC;

```

7. (0,5) Qual seria a instrução SQL adequada para realizar uma carga de dados em uma tabela nomeada de **tb_exemplo** a partir de um arquivo texto identificado como **teste.txt**, o qual utiliza “;” (**ponto e vírgula**) como delimitador. Suponha que esse arquivo texto esteja armazenado em um diretório qualquer (por exemplo: **c:\temp**).

```
COPY tb_exemplo
FROM 'c:/temp/teste.txt'
USING DELIMITERS ';';
```

8. (1,0) Suponha que exista uma tabela **tb_pedido** no banco de dados nomeado de **UNIFACEF**. Qual seria a instrução SQL utilizada para criar uma tabela nomeada de **tb_pedido_copia** extraindo a estrutura completa com suas respectivas tuplas (simultaneamente) da **tb_pedido**?

```
CREATE TABLE tb_pedido_copia
AS
SELECT *
FROM tb_pedido;
```

9. (0,5) Elabore uma consulta que produza as seguintes informações para cada empregado: <nome do empregado> recebe <salário> mensalmente, mas deseja <salário multiplicado por 3>. Insira um *label* “Salário dos Sonhos” para a coluna.

```
nome || 'recebe' || 'R$' || salario || ' mensalmente, mas deseja ' ||
'R$' || salario * 3 || ' ' "Salário dos Sonhos"
FROM tb_empregado;
```

10. (0,5) Realize a conexão com o *Banco de Dados (DataBase)* nomeado de **AVALIACAO_NP1** via linha de comando (*psql – shell/prompt*). Considere as seguintes informações abaixo:

- a. SGBD PostgreSQL versão 9.x
- b. Porta de comunicação: 5433
- c. Usuário: postgres
- d. Servidor: 127.0.0.1

```
psql -h 127.0.0.1 -p 5433 -d "AVALIACAO_NP1" -U postgres -W
```

11. (0,5) Por meio da instrução SQL, identifique o tamanho atual da tabela intitulada de “**tb_funcionario**”.

```
Select pg_relation_size('tb_funcionario');
```

12. (0,5) Por meio da instrução SQL, juntamente com o uso do catálogo pertinente ao *PostgreSQL*, liste o(s) nome(s) do(s) banco(s) de dado(s) (*databases*) existente(s) no servidor de banco de dados.

```
SELECT datname
FROM pg_database;
```

13. (0,5) Um Analista de Sistemas realizou a seguinte instrução SQL:

```
SELECT COUNT(*)
FROM tb_produtos
WHERE (fornecedor NOT IN (1,4,9))
AND (preco BETWEEN 97 AND 450);
```

Considere a tabela *tb_produtos*, a seguir:

codigo integer	produto character varying(15)	fornecedor integer	preco integer
1	Scanner	1	750
2	Teclado	7	95
4	Tonner	3	120
7	Monitor	1	450
9	Pendrive	7	25
14	Mouse	4	32

Assinale a alternativa que apresenta o valor que a instrução SQL irá retornar:

- (A) 0
- (B) 1**
- (C) 2
- (D) 3
- (E) 4

Resposta correta: B

14. (0,5) Quanto ao SQL (*Structured Query Language*), analise as afirmativas abaixo, dê valores Verdadeiro (V) ou Falso (F) e assinale a alternativa que apresenta a sequência correta (de cima para baixo):

- () DML é um subconjunto da linguagem SQL que é utilizado para realizar inclusões, alterações e exclusões de dados presentes em registros.
- () Os comandos básicos da DDL são poucos: INSERT, UPDATE e DELETE.
- () Duas palavras-chaves da DCL: GRANT e REVOKE.

- (A) V – V – V
- (B) V – V – F
- (C) V – F – V**
- (D) F – V – V
- (E) F – F – F

Resposta correta: C

15. (0,5) Selecione uma resposta correta em relação à seguinte consulta:

```
SELECT *  
FROM tb_empregado e  
INNER JOIN tb_departamento d ON (d.id_departamento = e.id_departamento)  
INNER JOIN tb_localizacao l ON (l.id_localizacao = d.id_localizacao);
```

- (A) Unir três tabelas não é permitido
- (B) Um erro de relacionado à junção é gerado
- (C) Um produto cartesiano é gerado
- (D) A cláusula INNER JOIN ... ON pode ser usada para junções entre diversas tabelas**
- (E) Nenhuma das alternativas anteriores

Resposta correta: D